

tcb/box tcb/boxSect
tcb/box/title tcb/box/titleCaption

Clase Algoritmos sobre Grafos

Universidad de Buenos Aires

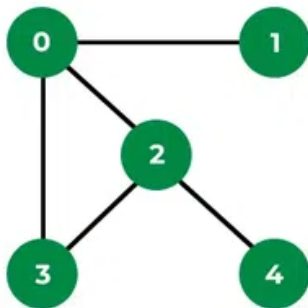
Definición DFS

DFS es un algoritmo recursivo que sigue la idea de backtracking para poder recorrer todos los nodos. Lo podemos usar tanto para grafos dirigidos como para no dirigidos.

Vamos a recorrer en profundidad (Depth): siempre vamos hasta el final de la rama y de ahí subimos.

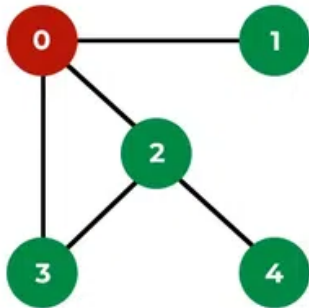
Ejemplo DFS

Tomamos un grafo de ejemplo con 5 nodos. Verde es no visitado, rojo es visitado.



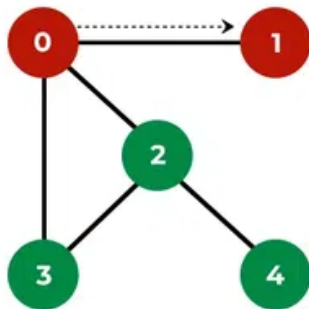
Ejemplo DFS

Arrancamos desde el 0 y lo marcamos como visitado. Ahora podemos recorrer cualquiera de los adyacentes al 0: 1, 2 o 3.



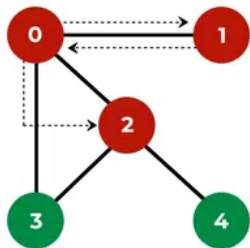
Ejemplo DFS

Visitamos el 1 y ahora podemos visitar a los adyacentes del 1 no visitados.



Ejemplo DFS

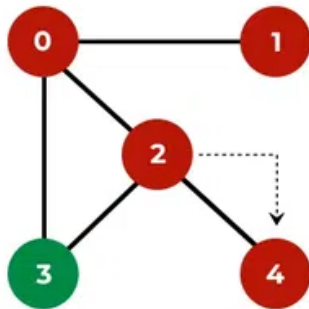
Como no hay adyacentes del 1 no visitados vuelve hacia arriba y sigue con los adyacentes no visitados del 0.



Ahora puedo seguir por cualquiera de los adyacentes del 2, que son el 3 o el 4.

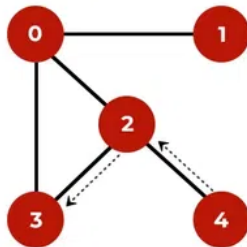
Ejemplo DFS

Visito el 4



Ejemplo DFS

Por el 4 no puedo seguir así que vuelvo al 2 y sigo en su lista de adyacentes, por lo que visito al 3.



Ahora vuelvo al 0 y ya no tengo más adyacentes que no fueron visitados.

La complejidad de DFS es $O(m + n)$ ya que recorro todos los nodos una sola vez y reviso las aristas también una sola vez (aunque puede que no recorra todas).

Sin agregarle complejidad al algoritmo de DFS podemos pedirle que nos devuelva el árbol o bosque (como un vector de padres) y la lista de backedges, que son las aristas que generan ciclos.

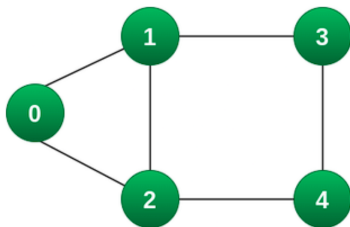
Definicion BFS

Con BFS también vamos a recorrer todos los nodos pero en lugar de recorrer en profundidad va recorriendo a lo ancho. Se suele implementar iterativo.

La idea del algoritmo es arrancar en algún nodo y recorrer todos sus vecinos, luego los vecinos de sus vecinos...

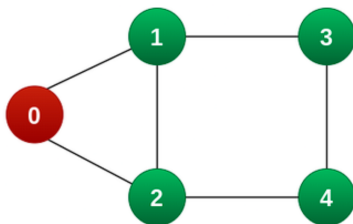
Ejemplo BFS

Tomemos como ejemplo este grafo y comenzamos a recorrer desde el nodo 0.



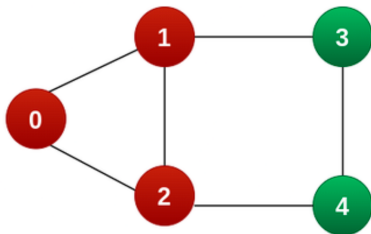
Ejemplo BFS

Visitamos el 0 y vemos cuáles son sus vecinos



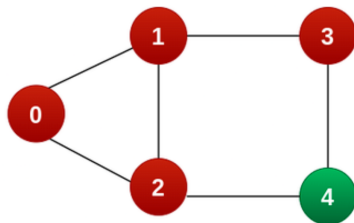
Ejemplo BFS

Visitamos ambos vecinos y luego vamos a ver los adyacentes al 1.



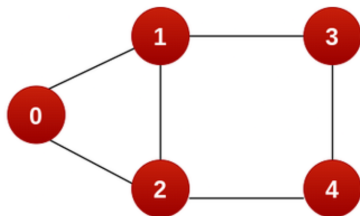
Ejemplo BFS

El 3 era el único adyacente al 1 que no había recorrido.



Ejemplo BFS

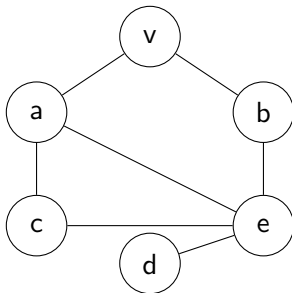
Sigo con los adyacentes del 2.



Luego el algoritmo se fijará en los vecinos del 3 y 4 pero como no quedan vecinos sin recorrer terminará la ejecución.

La complejidad de BFS también es **$O(m+n)$** , al igual que DFS.
Nos devuelve un árbol v -geodésico, siendo v el nodo desde el cual corremos el algoritmo. Puede devolver también las distancias de v a todos en el grafo (que son iguales que en el árbol ya que es v -geodésico!!)

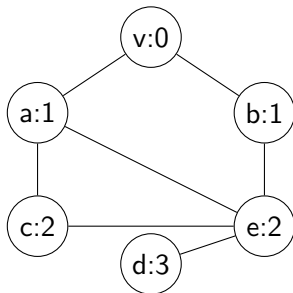
Necesitamos definir la *distancia* entre dos nodos en un grafo. Por ejemplo:



¿A qué distancia está cada nodo de v?

Árbol v-Geodésico

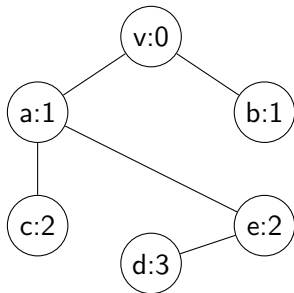
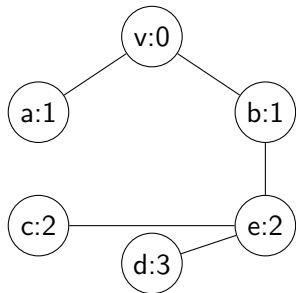
Necesitamos definir la *distancia* entre dos nodos en un grafo. Por ejemplo:



Un árbol v-geodésico es un sub-grafo que mantiene los nodos, que es un árbol y, además, que mantiene sus distancias al nodo distinguido v .

Árbol v-Geodésico

¿Cuál de estos árboles es v-geodésico?



¿Cómo nos damos cuenta rápido?

Ejercicio 1

Ejercicio 1: chequeo de conectividad

¿Cómo podemos ver que un grafo es conexo? Es decir, dado $G = (V, E)$, queremos chequear si $\forall u, w \in V, \exists$ camino entre u y w , o que $d(u, w) = k$ finito.

Ejercicio 1

Podríamos correr alguno de los algoritmos de recorrido y ver si todos los nodos fueron visitados (usando el vector `visitados`). Para esto usamos un código aún más simple que el visto en la teórica ya que si no puedo llegar a un nodo w corriendo DFS (o BFS) desde otro nodo v no me interesa correr de vuelta el algoritmo desde w , sino que ya sé que no es conexo. Si lo queremos resolver con DFS, podríamos usar esta implementación:

```
vector<vector<int>> aristas;
vector<bool> visitado;

void dfs(int v) {
    visitado[v] = true;
    for (int u : aristas[v]) {
        if (!visitado[u])
            dfs(u);
    }
}
```

Ejercicio 1

Ahora... ¿Cómo podemos ver si es conexo o no, luego del recorrido?
Podemos recorrer el vector de visitados y analizar si están todos los valores en *true*.

```
//Continuacion del codigo de arriba...

//Arranco el recorrido por el primer vertice
dfs(FIRST_NODE)

//Compruebo el vector de visitados
for(int i = 0; i < visitados.size(); i++){
    // Si encuentro alguno no visitado,
    // el grafo no es conexo
    if (!visitados[i]) return false;
}

// Si estan todos visitados, el grafo es conexo
return true;
```

Ejercicio 2

Ejercicio 2: componentes conexas

Dar un algoritmo que dado un grafo devuelva la cantidad de componentes conexas que tiene.

Definition

Dos vértices u, v pertenecen a la misma **componente conexa** si existe un camino entre ellos.

Ejercicio 2

Podríamos resolverlo con la implementación usual de DFS o BFS agregando un contador de componentes conexas o bien usar la implementación del ejercicio anterior y agregarle algo así:

```
int componentes = 0;
for (int i = 0; i < n; i++) {
    if (!visitado[i]) {
        componentes++;
        recorroAPartirDelVertice(i);
    }
}
```

Donde `recorroAPartirDelVertice` es la implementación del ejercicio anterior de DFS (o una similar de BFS).

Ejercicio 3

Chequeo de bipartito

Nos contrató el Chiqui Tapia para armar dos equipos para la selección (uno titular y uno suplente) de manera tal que no haya peleas internas en cada equipo.

Para esto nos dio una lista con los N jugadores y los C pares de conflictos (A,B) , que significan que A está peleado con B . Se quiere modelar el problema como un problema de grafos, en el que respondamos si es posible armar dos equipos sin conflictos internos.

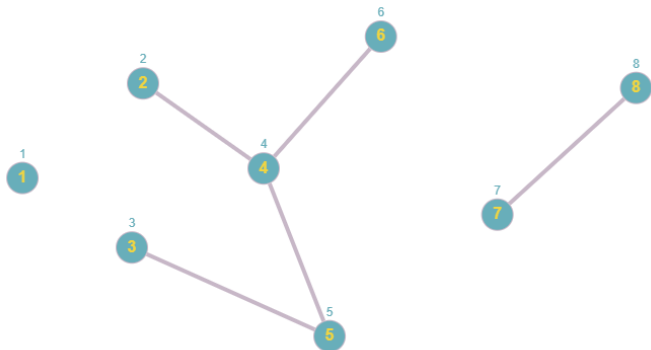
Ejercicio 3

Primero modelemos el problema como un grafo. Sea $G = (V, E)$.

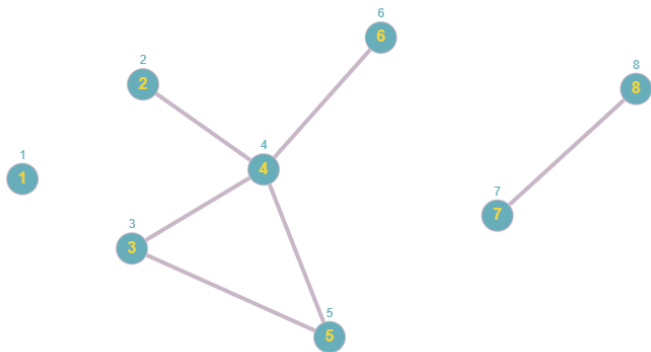
- ¿Quiénes van a ser los vértices de V ? Los N jugadores
- ¿Quiénes van a ser las aristas de E ? Los conflictos

Ejercicio 3

Con éste se puede?



Y con éste?



Ejercicio 3

Idea:

- Puede haber jugadores sin conflictos, entonces la solución tiene que preservarse para cada **componente conexa**.
- Recordemos que dos vértices u, v pertenecen a la misma **componente conexa** si existe un camino entre ellos.

Ejercicio 3

- Lo que queremos es que cada componente conexa sea un **grafo bipartito**.

Definition

Un grafo es **bipartito** si existen dos conjuntos V_1 y V_2 tales que $V_1 \cup V_2 = V$, $V_1 \cap V_2 = \emptyset$ y $\forall v \in V_1, (u, v) \in E \implies u \in V_2$, y análogamente para V_2 .

Es decir que los vértices en V_1 sólo son adyacentes a los de V_2 y los de V_2 sólo son adyacentes a los de V_1 .

Ejercicio 3

- En resumen, quiero verificar si todas las componentes conexas de G son bipartitas.
- Puedo usar DFS e ir pintando con dos colores distintos. Si dos nodos consecutivos debo pintarlos con el mismo color, entonces el grafo no es bipartito.

Además, tenemos el lema:

Definition (Lema)

Sean $u, v \in V : (u, v) \in E$.

G es bipartito $\iff$ no hay ningún ciclo impar

Solución

```
vector<vector<int>> aristas;  
vector<int> color;    // -1 = sin visitar, 0 y 1 = los dos colores  
  
bool dfs(int v, int c) {  
    color[v] = c;  
    for (int u : aristas[v]) {  
        if (color[u] == -1) {  
            if (!dfs(u, 1 - c))    // pinto al vecino del color  
                                    opuesto  
                return false;  
        } else if (color[u] == c) {    // vecino con mi mismo color  
            -> no bipartito  
            return false;  
        }  
    }  
    return true;  
}  
}
```

```
bool esBipartito(int n) {  
    color.assign(n, -1);  
    for (int v = 0; v < n; v++) {  
        if (color[v] == -1) {  
            if (!dfs(v, 0))  
                return false;  
        }  
    }  
    return true;  
}
```

Ejercicio 4

Ejercicio 4: detectar aristas puente

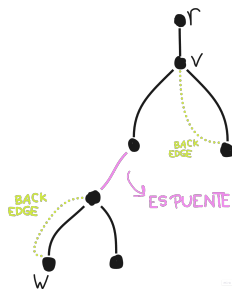
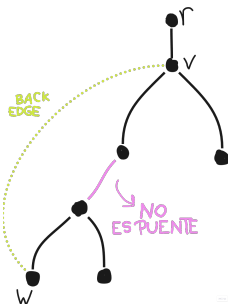
Dado G conexo:

- a Demostrar que vw es un puente de G si y solo si vw no pertenece a ningún ciclo de G .
- b Dar un algoritmo lineal basado en DFS para encontrar todas las aristas puente de G .

Ejercicio 4

A tener en cuenta:

- Una arista es puente si al sacarla aumenta la cantidad de componentes conexas
- Una back-edge nunca puede ser puente
- Las aristas que son puentes son aquellas tree-edges que no tienen una back-edge que las “cubra”



Ejercicio 4

Empecemos con la demostración. Vamos a probar primero la ida y luego la vuelta.

IDA: vw es un puente de $G \implies vw$ no pertenece a ningún ciclo de G

Vamos a probarlo por absurdo. Para eso supongamos que vw es un puente de G , y que pertenece a un ciclo de G

- Como vw pertenece a un ciclo, hay un camino de v a w que no usa la arista vw
- Por lo tanto, si quitamos vw del grafo, v y w siguen conectados por ese otro camino.
- Pero esto contradice el hecho de que vw es un puente, ya que por definición, eliminar un puente aumenta el número de componentes conexas. **ABSURDO**

Ejercicio 4

VUELTA: vw no pertenece a ningún ciclo de $G \implies vw$ es un puente de G
Lo probamos por contrarrecíproco ($\neg B \implies \neg A$). Entonces queremos probar que:

vw no es un puente de $G \implies vw$ pertenece a algún ciclo de G

- Si vw no es un puente, significa que al eliminar la arista vw , el grafo $G' = G - \{vw\}$ sigue siendo conexo.
- Es decir, existe un camino alternativo P que une v con w y no utiliza la arista vw .
- Si en G' agregamos la arista vw nuevamente, obtenemos el camino P junto con vw , que en conjunto forman un ciclo.
- Por lo tanto, vw pertenece a un ciclo en G , como queríamos ver

Ejercicio 4

Ahora sí, arranquemos con el inciso b.

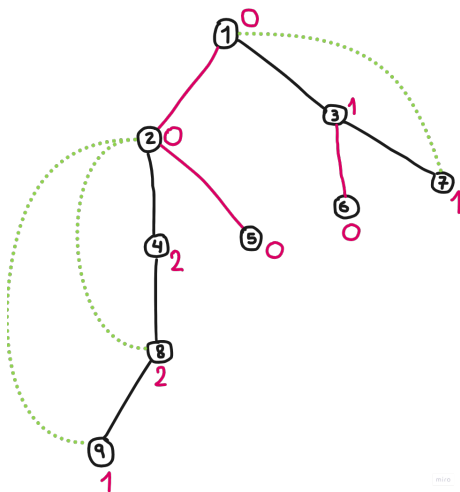
Primero debemos convencernos de que la cantidad de backedges que cubren la arista entre v y su padre se puede calcular de la siguiente manera:

$$\begin{aligned} \text{cubren}(v) = & \text{backEdgesConExtremoInferiorEn}(v) \\ & - \text{backEdgesConExtremoSuperiorEn}(v) \\ & + \sum_{w \in \text{hijos}(v)} \text{cubren}(w) \end{aligned}$$

Veamos un ejemplo.

Ejercicio 4

Los números al lado de los nodos indican el valor de cubren en dicho nodo. Si cubren de un nodo v es 0 significa que la arista de v al padre es puente



Ejercicio 4

Ahora corremos un DFS que de alguna manera calcule para cada nodo `backConExtremoInferiorEn` y `backConExtremoSuperiorEn`.
Puede ser algo así:

```
vector<vector<int>> treeEdges; //init en []
...
void dfs(int v, int p = -1) {
    estado[v] = EMPECE_A_VER;
    for (int u : aristas[v]) {
        if (estado[u] == NO_LO_VI) {
            treeEdges[v].push_back(u);
            dfs(u, v);
        } else if (u != p && estado[u] != TERMINE_DE_VER){
            backConExtremoInferiorEn[v]++;
            backConExtremoSuperiorEn[u]++;
        }
    }
    estado[v] = TERMINE_DE_VER;
}
```

En este DFS si ya pasé por un nodo y no es el padre entonces sumo una `backedge`.

Notar que en esta implementación de DFS se asume que los nodos tienen tres estados: no lo vi, empecé a ver y terminé de ver

Ejercicio 4

Hacemos un algoritmo que calcule cubren. Por la fórmula anterior, para calcular cubren de un nodo tenemos que tener la de los hijos ya calculada.

```
vector<int> memo; //init en -1
int cubren(int v, int p = -1) {
    if (memo[v] != -1) return memo[v];
    int res = 0;
    res += backConExtremoInferiorEn[v];
    res -= backConExtremoSuperiorEn[v];
    for (int hijo : treeEdges[v]) if (hijo != p)
        res += cubren(hijo, v);
}

memo[v] = res;
return res;
}
```

Ejercicio 4

Y ahora finalmente solo necesito calcular cuántos puentes nos quedan, que es la cantidad de nodos v tal que $\text{cubren}(v) = 0$.

```
int puentes = 0;
for (int i = 0; i < n; i++) {
    if (cubren(i) == 0) {
        puentes++;
    }
}
```

OJO si G no fuera conexo nos faltaría restar la cantidad de componentes ya que como vimos en los ejemplos la raíz de los árboles también tiene $\text{cubren} = 0$.

$\text{puentes} -= \text{componentesConexas}$

Ejercicio 4

Complejidad?

- DFS: **$O(n+m)$**
- cubren: **$O(n)$** ya que como máximo llenamos el vector de memoización
- puentes: **$O(n)$** porque recorro el vector cubren que tiene longitud cant nodos

¿Nos tomamos un break?



Ejercicio 5

Calles bidireccionales

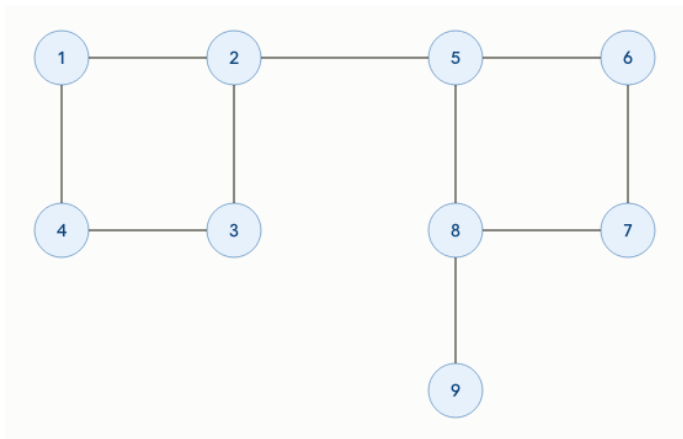
Una ciudad tiene N intersecciones (esquinas) y M calles bidireccionales. Hoy se puede llegar de cualquier intersección a cualquier otra. El intendente quiere convertir la mayor cantidad posible de calles a mano única, con la condición de que se siga pudiendo llegar de cualquier intersección a cualquier otra respetando los sentidos. Determinar qué calles pueden convertirse y asignarles una dirección válida.

Ejercicio 5

Empecemos con el modelado:

- N intersecciones $\rightarrow$ *Nodos*
- M calles $\rightarrow$ *Aristas*

Ejercicio 5



En este ejemplo, ¿Qué calles se pueden hacer unidireccionales y cuáles no?

Ejercicio 5

Algoritmo:

- Corremos el algoritmo de puentes antes ($O(N+M)$)
- Cada arista puente queda bidireccional
- Para cada arista no puente, la dirigimos según el mismo DFS: las de árbol hacia el hijo, las backedges hacia el ancestro.

Ejercicio 6: Puentes vs. puntos de corte

Puentes vs. puntos de corte

Sea G un grafo con al menos tres vértices.

- v es *punto de corte* si $G \setminus v$ tiene más componentes conexas que G .
- (v, w) es *puente* si $G \setminus (v, w)$ tiene más componentes conexas que G .

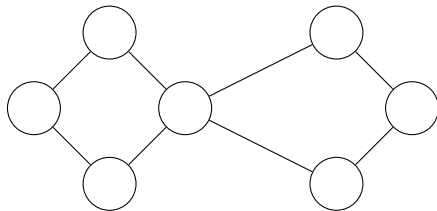
Decidir si cada afirmación es verdadera o falsa.

Inciso (a)

Si G es conexo y no tiene puentes, entonces G tiene exactamente un ciclo.

Falsa.

Contraejemplo: dos ciclos que comparten un vértice.



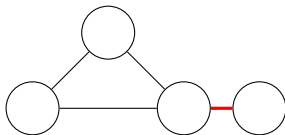
El grafo es conexo y ninguna arista es puente, porque toda arista pertenece a algún ciclo. Pero tiene más de un ciclo.

Inciso (b)

Si G es conexo y tiene exactamente un ciclo, entonces G no tiene puentes.

Falsa.

Contraejemplo: un triángulo con una hoja.



Tiene exactamente un ciclo: el triángulo. Pero la arista roja es puente.

Inciso (c)

Si G es conexo y no tiene puntos de corte, entonces G no tiene puentes.

Verdadera.

Lo probamos por absurdo.

Supongamos que G no tiene puntos de corte, pero tiene un puente (v, w) . Al remover (v, w) , el grafo queda separado en dos componentes conexas:

$$A \text{ y } B,$$

con $v \in A$ y $w \in B$.

Inciso (c): demostración

Como G tiene al menos tres vértices, alguna de las dos componentes tiene al menos dos vértices.

Caso 1: $|A| \geq 2$.

Al remover v , queda algún vértice de $A \setminus \{v\}$ y queda toda la componente B .

No puede haber aristas entre $A \setminus \{v\}$ y B , porque entonces (v, w) no sería puente.

Entonces $G \setminus v$ es desconexo. Por lo tanto, v es punto de corte.

Contradicción.

Inciso (c): segundo caso

Caso 2: $|A| = 1$.

Entonces, como G tiene al menos tres vértices, se cumple $|B| \geq 2$.

Al remover w , queda el vértice v y queda algún vértice de $B \setminus \{w\}$.

No hay aristas entre esos conjuntos, porque (v, w) era puente.

Entonces $G \setminus w$ es desconexo. Por lo tanto, w es punto de corte.

Contradicción.

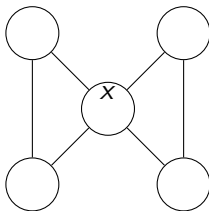
Luego G no tiene puentes.

Inciso (d)

Si G es conexo y no tiene puentes, entonces G no tiene puntos de corte.

Falsa.

Contraejemplo: el grafo “moño”, dos ciclos que comparten un único vértice.



No tiene puentes, porque toda arista pertenece a un ciclo. Pero x es punto de corte.

Ejercicio 7: Árbol Geodésico Mínimo

Árbol geodésico de peso mínimo

Diseñar un algoritmo de tiempo $O(n + m)$ que, dado un grafo conexo G con pesos en sus aristas y un vértice v , determine el árbol de menor peso de entre todos los árboles v -geodésicos de G , es decir de los árboles v -geodésicos considerando el grafo sin pesos. El peso $w(T)$ de un árbol T es igual a la suma de los pesos de las aristas que pertenecen al árbol.

Demostrar que el algoritmo propuesto es correcto.

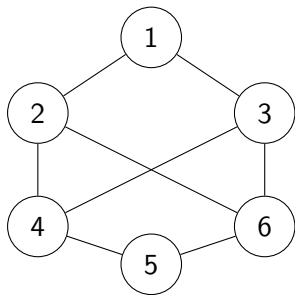
Ejercicio Árbol Geodésico Mínimo - Intuición

Intuición:

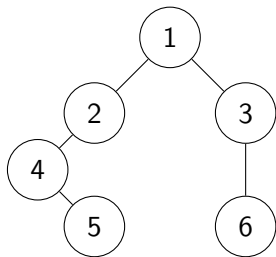
- hacemos el precálculo de las distancias de alguna forma
- el padre de cada nodo en el árbol tiene que ser alguno que acorte la distancia en 1
- de todos los árboles v -geodésicos, armamos el de peso mínimo

Ejercicio Árbol Geodésico Mínimo

Primero realizamos un BFS desde v en el grafo sin considerar los pesos de las aristas. De esta manera obtenemos, para cada $u \in V(G)$, su distancia $d[u] = d_G(v, u)$.



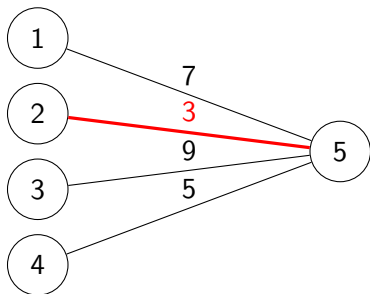
Grafo original



Árbol BFS desde 1

Ejercicio Árbol Geodésico Mínimo

Luego, para cada vértice $u \neq v$, elegimos entre todos sus vecinos w tales que $d[w] = d[u] - 1$ aquel para el cual la arista (w, u) tiene menor peso.



Es decir, elegimos

$$p(u) \in \arg \min_{\substack{w \in N(u) \\ d[w] = d[u] - 1}} c(w, u),$$

y agregamos la arista $(p(u), u)$ al árbol T .

Demostración Árbol Geodésico Mínimo

Veamos primero que T es un árbol v -geodésico.
Para cada $u \neq v$, la arista elegida $(p(u), u)$ cumple

$$d[p(u)] = d[u] - 1.$$

pues es una de las condiciones que le pedimos. Así que la distancia mínima se mantiene. Y además estamos agregando aristas entre niveles del árbol BFS y sólo 1 por cada vértice, así que se mantiene la conexitud y la ausencia de ciclos, por lo que sigue siendo un árbol.

Demostración Árbol Geodésico Mínimo

Probemos ahora que T tiene peso mínimo.

Sea T' cualquier árbol v -geodésico. Para cada vértice $u \neq v$, sea $p'(u)$ el vértice anterior a u en el único camino de v a u en T' .

Como T' es v -geodésico, $d[p'(u)] = d[u] - 1$. Por construcción, $p(u)$ fue elegido entre todos los vecinos de u de la capa anterior con peso mínimo.

Por lo tanto,

$$c(p(u), u) \leq c(p'(u), u)$$

para todo $u \neq v$.

Como cada arista de un árbol enraizado en v corresponde al padre de exactamente un vértice distinto de v , tenemos

$$w(T) = \sum_{u \neq v} c(p(u), u) \leq \sum_{u \neq v} c(p'(u), u) = w(T').$$

Por lo tanto, T es un árbol v -geodésico de peso mínimo.

Presentación del taller

- Fecha de entrega: miércoles 9 de septiembre (recuperatorio miércoles 16 de septiembre)
- Completar el form, y unirse al grupo de Codeforces
- Para aprobar: dos ejercicios marcados con AC (respuesta correcta) en el juez.
- Si hacen los tres correctamente y aprueban los parciales, +0.33 puntos
- Vale repetir entregas de problemas todas las veces que necesiten
- ¡Hagan todas sus entregas privadas!
- Prohibido usar código de otras personas (compañeres o copiado y pegado de internet)
- Prohibido entregar código generado con IA